

Министерство образования и науки Российской Федерации  
Федеральное государственное автономное образовательное учреждение  
высшего образования

САНКТ-ПЕТЕРБУРГСКИЙ НАЦИОНАЛЬНЫЙ  
ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ ИНФОРМАЦИОННЫХ  
ТЕХНОЛОГИЙ, МЕХАНИКИ И ОПТИКИ

Факультет программной инженерии и компьютерных технологий

Отчёт по лабораторной работе №1

**Выполнил:** Семикозов Ян Леонидович

**Группа:** Р4119

**Преподаватель:** Табунщик Сергей Михайлович

# 1 Цель работы

Получить общее представление о системе команд RISC-V путём решения несложной прикладной задачи – найти наименьший элемент на диагонали в матрице  $6 \times 6$ .

# 2 Ход работы

В данной работе используется архитектура RISC-V с набором команд RV32I. В соответствии с *соглашением о вызовах функций RISC-V* первые четыре аргумента функции передаются через регистры **a0–a3**, а результат возвращается в регистре **a0**. Ниже приведена сводная таблица используемых регистров и их назначение в программе (таблица 1).

Таблица 1. Назначение используемых регистров

| Регистры | Номер (x#) | Назначение в коде                                         |
|----------|------------|-----------------------------------------------------------|
| ra       | x1         | Адрес возврата из функции                                 |
| sp       | x2         | Указатель на вершину стека                                |
| s0       | x8         | Базовый адрес матрицы                                     |
| s1       | x9         | Текущее минимальное значение                              |
| s2       | x10        | Индекс цикла i                                            |
| t0       | x5         | Граница цикла                                             |
| t1       | x6         | Вычисление смещения <code>matrix[i][i]</code>             |
| t2       | x7         | Текущий элемент <code>matrix[i][i]</code>                 |
| a0       | x10        | Первый аргумент функции / возвращаемое значение           |
| a1       | x11        | Второй аргумент функции / аргумент для <code>ecall</code> |
| a7       | x17        | Код системного вызова ( <code>ecall</code> )              |

Прежде чем описывать функциональную часть программы разместим входные и неизменяемые данные в сегменте `.data`.

```
1      .data
2  matrix:
3      .word 12, 7, 31, 4, 5, 6
4      .word 8, 10, 2, 11, 1, 9
5      .word 14, 15, 6, 13, 12, 7
6      .word 16, 17, 18, 5, 19, 20
7      .word 21, 22, 23, 24, 9, 25
8      .word 26, 27, 28, 29, 30, 10
9
10     minelemstr:
11     .asciiiz "min element: "
```

### Листинг 1. Код программы на языке ассемблера RISC-V

Все инструкции программы разместим в сегменте `.text`; для глобальной видимости функций используется метка `.globl`. В функции `findMinElement` вначале создадим стековый фрейм для сохранения регистров, которые будут изменяться. При этом выделим 16 байт на стек и сохраним в него регистры `ra`, `s0`, `s1` и `s2`. При завершении функции восстановим сохранённые значения регистров и освободим стековый фрейм.

```

1      .text
2      .globl findMinElement
3      findMinElement:
4          addi sp, sp, -16
5          sw ra, 12(sp)
6          sw s0, 8(sp)
7          sw s1, 4(sp)
8          sw s2, 0(sp)
9
10         mv s0, a0                # s0 = matrix address
11
12         lw s1, 0(s0)             # s1 = min = matrix[0][0]
13
14         li s2, 1                 # i = 1
15
16     loop:
17         li t0, 6
18         bge s2, t0, endloop      # if i >= 6 → exit
19
20         # matrix[i][i] offset = (i*6 + i)*4 = i*7*4
21         li t1, 7
22         mul t1, t1, s2           # t1 = i * 7
23         slli t1, t1, 2           # t1 *= 4
24         add t1, s0, t1           # t1 = &matrix[i][i]
25
26         lw t2, 0(t1)             # t2 = matrix[i][i]
27
28         blt t2, s1, updateMin    # if matrix[i][i] < min
29         j next_i
30
31     updateMin:
32         mv s1, t2
33
34     next_i:
35         addi s2, s2, 1
36         j loop
37
38     endloop:
39         mv a0, s1                # return min
40
41         lw ra, 12(sp)
42         lw s0, 8(sp)
43         lw s1, 4(sp)
44         lw s2, 0(sp)
45         addi sp, sp, 16
46         ret

```

Листинг 2. Код функции *findMinElement* на языке ассемблера RISC-V

Ниже приведён код функции `main`, которая вызывает функцию поиска минимального элемента и выводит результат на экран с использованием системных вызовов. Код системного вызова записывается в регистр `a7`, а аргумент в `a1`:

- Код 1 – вывод целого числа.
- Код 4 – вывод строки.
- Код 11 – вывод символа.

```
1  main:
2      addi sp, sp, -16
3      sw ra, 12(sp)
4
5      la a0, matrix           # argument: pointer to matrix
6      call findMinElement     # a0 <- result
7
8      mv t0, a0               # save result in t0
9
10     # print string
11     la a1, minelemstr
12     li a0, 4
13     ecall
14
15     # print integer
16     mv a1, t0
17     li a0, 1
18     ecall
19
20     # print newline
21     li a1, 10
22     li a0, 11
23     ecall
24
25
26     lw ra, 12(sp)
27     addi sp, sp, 16
28     li a0, 0
29     ret
30
```

Листинг 3. Код функции *main* на языке ассемблера RISC-V

Ниже в листинге 4 приведён код программы на языке C, решающей ту же задачу.

```

1  #include <stdio.h>
2
3  #define N 6
4  #define M 6
5
6  int findMinElement(int matrix[N][M])
7  {
8      int min = matrix[0][0];
9
10     for (int i = 1; i < N && i < M; i++) {
11         if (matrix[i][i] < min) {
12             min = matrix[i][i];
13         }
14     }
15
16     return min;
17 }
18
19 int main()
20 {
21     int matrix[N][M] = {
22         { 12, 7, 31, 4, 5, 6 },
23         { 8, 10, 2, 11, 1, 9 },
24         { 14, 15, 6, 13, 12, 7 },
25         { 16, 17, 18, 5, 19, 20 },
26         { 21, 22, 23, 24, 3, 25 },
27         { 26, 27, 28, 29, 30, 4 }
28     };
29
30     int min = findMinElement(matrix);
31
32     printf("min element: %d\n", min);
33
34     return 0;
35 }

```

Листинг 4. Код программы на С

Сборка и запуск программы осуществлялись с использованием собранного кросс-компилятора *RISC-V* и эмулятора *QEMU*. Ниже приведены команды, использованные для сборки и запуска программы:

```
make -C src && qemu-riscv32 src/main.elf
```

Далее код был переписан на языке ассемблере *RISC-V* и проверен в симуляторе *Venus*.

### 3 Выводы

В ходе выполнения лабораторной работы изучены базовые инструкции архитектуры RISC-V. Реализована программа на языке C и её эквивалент на языке ассемблера. Ознакомились с процессом сборки и запуска программ для архитектуры RISC-V.